

Project Report: Unified Coffee Ordering System (UCOS)

Genesis Anne Villar

San Diego State University - FALL 2023

CS-250: Intro to Software Systems

Professor Parker

December 8, 2023

TABLE OF CONTENTS

Design Documentation.....	4 - 25
Project Overview.....	4
Client Requirements.....	4
Introduction.....	4
Software Architecture Overview.....	5 - 6
Architectural Diagram	6
Design Implementation	6 - 13
UML Class Diagram.....	8
Software Description – Classes	9 - 10
Software Description – Attribution	10 - 11
Software Description – Operations	11 - 13
Verification Test Plan	14 - 17
Scope	14
Test Set 1: User Profile & Payment Management	14 - 16
Test Set 2: Coffee Order Processing & Statistics Generation	16 - 17
Data Management Strategy	18 - 22
Implementation Rationale	18
Statistics Database	18- 20
Inventory/Usage Database	20 - 21
Tradeoff Discussion	21 - 22
Possible Alternatives	22
Future Modifications	23

TABLE OF CONTENTS (cont.)

Security Analysis	24 - 25
Confidentiality	24
Integrity	24
Availability	25
User Manual	26 - 28
System Description	26
Application Functionality	26 - 27
Placing and Customizing Orders	26
Payment and Information Management	26
Order Management for Employees	26
Error Reporting and Logging	27
Accessing Analytics	27
Interpreting Reports	27
Inventory Management	27
System Requirements	28
Installation Guide	28
Life-Cycle Model Reflection	29 - 31
Description of Lifecycle	29 - 30
Effectiveness and Potential Modifications	30 - 31

DESIGN DOCUMENTATION

PROJECT OVERVIEW

CLIENT REQUIREMENTS

The Java Coffee Corporation, an international chain offering a diverse range of coffee products, has requested the creation of a unified coffee ordering system to address inconsistencies that have led to revenue loss and customer dissatisfaction. The system has to support smooth updates across all locations, a wide range of add-ons, and highly configurable coffee orders. In case of order problems or system flaws, it should also include error message capabilities. To monitor overall sales, drink popularity, and average order costs, and to maintain daily operational continuity, the company also needs tracking and data storage tools, as well as the ability to create and retrieve coffee orders via databases. The objective is to improve the ordering procedure and provide a uniform tracking and customer experience throughout all stores.

INTRODUCTION

This Software Design Specification (SDS) document provides a detailed overview of the Unified Coffee Ordering System (UCOS) software created to streamline the order process across all branches, ensuring a consistent and efficient customer experience. Features that were asked to be implemented in this system include a coffee creation tool, an ordering system, order tracking and analysis, and as well as data storage and retrieval. Through this integrated solution, we aspire to enhance the customer experience, facilitate easy order tracking, and streamline revenue management.

SOFTWARE ARCHITECTURE OVERVIEW

ARCHITECTURAL DIAGRAM

UCOS is a sophisticated system that integrates order processing with inventory management and business analytics to provide a seamless experience for both customers and staff. The SWA diagram maps out the digital ecosystem of this software in action. It is structured in three layers: the Infrastructure Layer, Business Logic Layer, and User Interface Layer. Namely, it highlights the structured flow of information– beneficial for customers and employees by optimizing order placements and business analytics.

INFRASTRUCTURE LAYER:

- **Cloud-based Server:** A server that connects with the whole coffee company database
- **Statistics Database:** Stores statistical data related to sales and inventory.
- **Inventory/Order Database:** Stores inventory and order records.

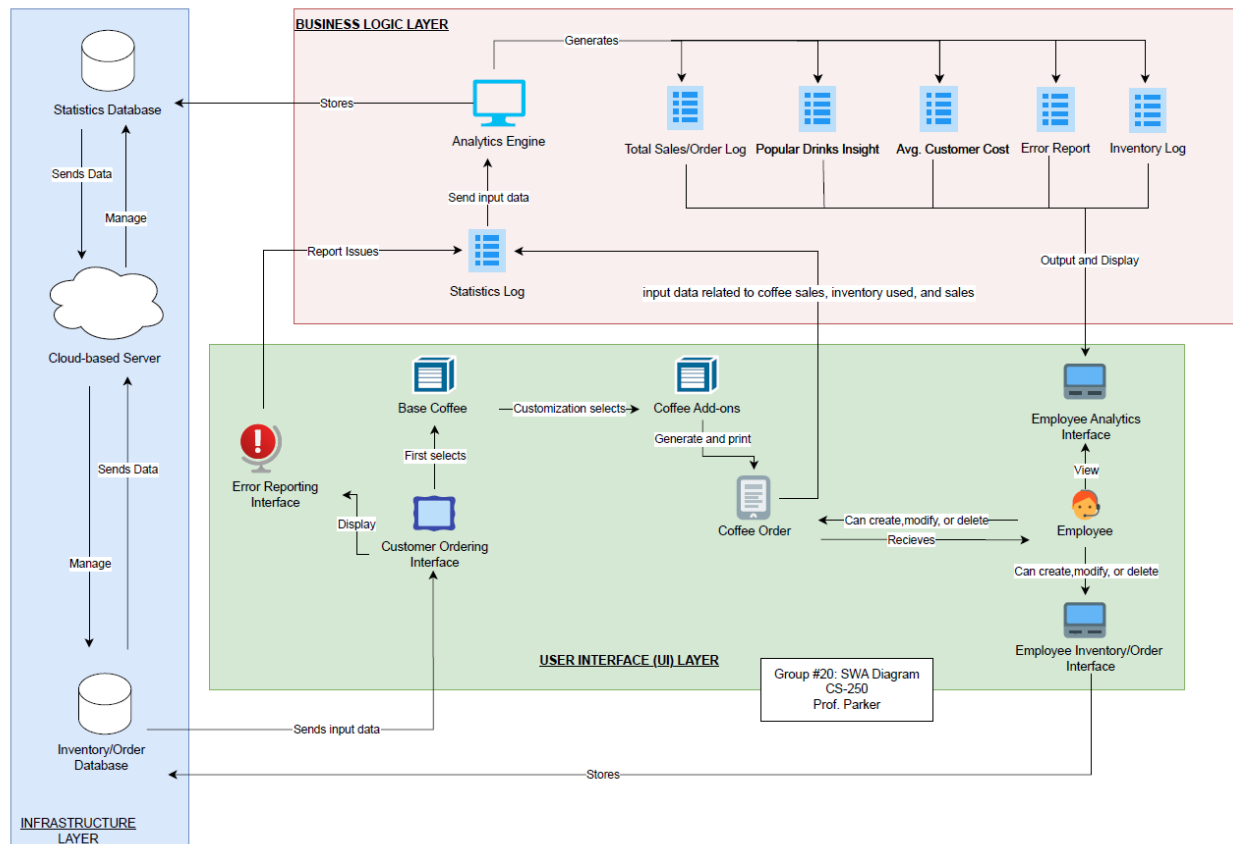
BUSINESS LOGIC LAYER

- **Analytics Engine:** Processes data inputted to provide various business insights
- **Statistics Log:** Logs all coffee orders and errors reported to be inputted
- **Various Reports:** Outputs of data analyses like sales, drink popularity, and inventory status

USER INTERFACE (UI) LAYER:

- **Error Reporting Interface:** Allows users to report system errors.
- **Customer Ordering Interface:** Allows customers to customize and place their coffee orders.
 - Based on Base Coffee and Coffee-addons

- **Coffee Order:** Represents a customer's finalized coffee selection.
- **Employee Analytics Interface:** A dashboard for employees to view business analytics.
- **Employee Inventory/Order Interface:** Manages inventory and orders by employees



DESIGN IMPLEMENTATION

The UML diagram for the Unified Coffee Ordering System is meticulously structured to encapsulate the complexities of the application while maintaining clarity in the representation of its components. Every class is designed with the right access modifiers, denoted by the '+' for public members and the '-' for private members, following the encapsulation concept that protects the attributes unique to each class.

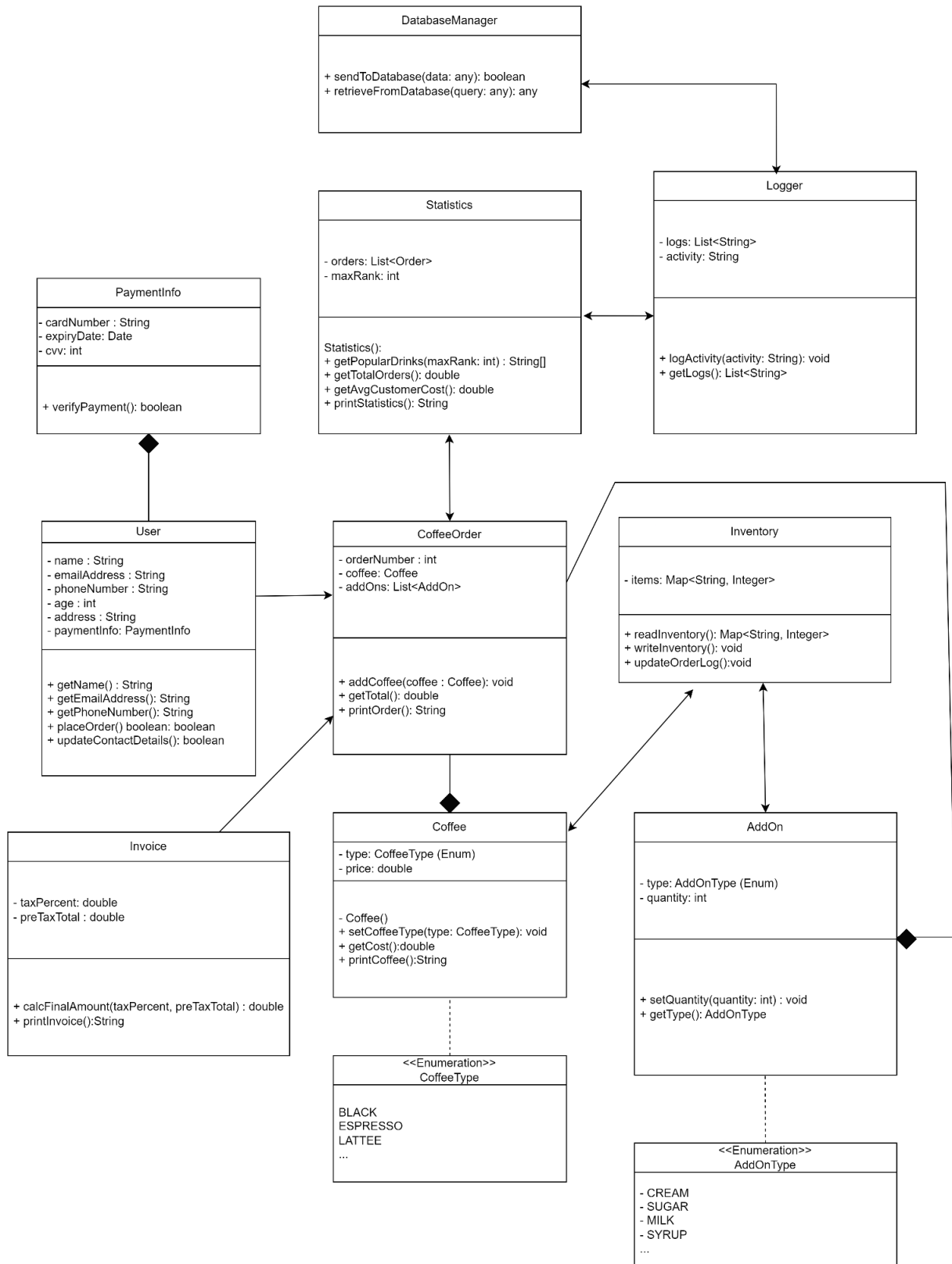
The relationships between classes are distinctly represented using a variety of connectors:

- Composition: solid diamond arrows, indicating a strong lifecycle dependency between related classes (i.e. *User* class and *PaymentInfo* class)
- Aggregation: open diamond arrows, signifying a whole-part relationship that does not imply ownership (i.e. *CoffeeOrder* class and its constituents *Coffee* and *AddOn*)
- Simple lines: either one-to-one or one-to-many associations (i.e. *User* and *CoffeeOrder*)
- Dashed arrows: dependencies (i.e. *Coffee* class relying on the *CoffeeType* enumeration)

The UML Diagram's architecture is built to have a clear division of responsibilities within the project. Financial transaction features, for example, are now divided into two classes: the “*Invoice*” class, which manages order cost details, and the “*Payment*” class, which focuses on payment transaction verification. This division improves system resilience and error handling.

Further granularity is introduced in the ordering process through the use of enumerations, specifically for the “*Coffee*” class and the “*AddOn*” classes. To illustrate this, enumerations for the “*Coffee*” class deal with specific coffee types (which can be added or removed), so each instance will represent different cost properties and different strings to print for the final coffee order. Enumerations for the “*AddOn*” class represent a specific add-on type, quantity, and cost for each instance of coffee created. These enumerations provide a structured method to represent varying types and quantities, imbuing each coffee creation instance with precise attributes and pricing.

UML CLASS DIAGRAM



SOFTWARE DESCRIPTION - CLASSES

- **DatabaseManager:** responsible for handling all interactions with the database. It provides methods to send data to and retrieve data from the database, ensuring that data management is centralized and consistent throughout the application.
- **Statistics:** responsible for all the data collected so the shop owner has an overview of how the business is going. This class holds a list of CoffeeOrder objects to compute and provide statistical insights on coffee orders based on that information, which can be printed out for employees to look at such data.
- **Logger:** used to maintain a log of system activities, which can help debug and track the sequence of operations. It supports adding activity logs and retrieving them.
- **PaymentInfo:** handles the sensitive information related to payment, such as card number and expiry date. It includes methods to verify the payment.
- **Invoice:** This class is responsible for the financial details of an order, such as tax percentage and pre-tax total. It calculates the final amount and prints the invoice.
- **User:** represents the customer using the system. It holds personal and contact details along with payment information and has methods to place an order and update contact details.
- **CoffeeOrder:** Manages the specifics of a coffee order, including order number, coffee details, and add-ons. It provides methods to add coffee to the order, get the total cost, and print the order.
- **Coffee:** This class defines a coffee item in the order, specifying its type and price. It includes methods to set and get coffee type and price, and print details of the coffee

- **AddOn:** Represents additional items that can be added to a coffee, such as cream or sugar. It provides methods to set the quantity and get the type of add-ons.
- **Inventory:** Manages the inventory of coffee-related items. It tracks item stock and includes methods for inventory and order log management.

SOFTWARE DESCRIPTION - ATTRIBUTION

- **Statistics:**
 - **orders (List<Order>)** - List of coffee orders for statistical reference
 - **maxRank (int)** - The maximum rank for calculating popular drinks
- **Logger:**
 - **logs (List<String>)** - A list of strings representing logged activities
 - **activity (String)** - The current activity being logged
- **PaymentInfo:**
 - **cardNumber (String)** - The payment card number
 - **expiryDate (Date)** - The expiration date of the payment card
 - **cvv (int)** - The card verification value
- **Invoice:**
 - **taxPercent (double)** - The tax percentage to be applied to the order
 - **preTaxTotal (double)** - The total cost before tax is applied
- **User:**
 - **name (String)** - The name of the user
 - **phoneNumber (String)** - The contact phone number of user
 - **emailAddress (String)** - The email address of user
 - **age (int)** - The age of the user

- **address (String)** - The physical address of the user
- **paymentInfo (PaymentInfo)** - The user's payment information of the user
- **CoffeeOrder:**
 - **orderNumber (int)** - The unique identification number for the specific order
 - **coffee (Coffee)** - The coffee item in the order
 - **addOns (List<AddOn>)** - A list of add-ons included in the order
- **Coffee:**
 - **price (double)** - The price of each type of coffee
 - **type (CoffeeType (Enum))** - The type of coffee ordered
- **AddOn:**
 - **type (AddOnType (Enum))** - The type of add-on
 - **quantity (int)** - The quantity of the add-on
- **Inventory:**
 - **items (Map<String, Integer>)** - A map of items with their corresponding stock numbers

SOFTWARE DESCRIPTION - OPERATIONS

DatabaseManager:

- **sendToDatabase(data: any): boolean** - Sends data to the database. Return true if data was successfully sent, false if an error occurred
- **retrieveFromDatabase(query: any): any** - Retrieves data from the database based on a query

Statistics:

- **Statistics():** Constructor for initializing the statistics object

- **getTotalOrders(): double** - Returns the total number of orders
- **getAvgCustomerCost(): double** - Calculates and returns the average cost orders
- **getPopularOrders(maxRank: int): String[]** - Retrieves a list of popular drinks up to a specified rank
- **printStatistics(): String** - Returns a string representation of the statistics for display or reporting

Logger:

- **logActivity(activity: String): void** - Records an activity to the log
- **getLogs(): List<String>** - Retrieves the list of logged activities

PaymentInfo:

- **verifyPayment(): boolean** - Validates the payment information. Return true if the payment is valid, false otherwise

Invoice:

- **calcFinalAmount(taxPercent, preTaxTotal): double** - Calculates the final amount of an order by applying the tax percentage to the pre-tax total
- **printInvoice(): String** - Generates and returns the string representation of the invoice

User:

- **getName(): String** - Retrieves the name of selected user
- **getEmailAddress(): String** - Retrieves the email address of selected user
- **getPhoneNumber(): String** - Retrieves the phone number of selected user
- **placeOrder(): boolean** - Submits a coffee order for the user. Returns true if the order is placed successfully, false otherwise

- **updateContactDetails(): boolean** - Updates the user's contact details. Returns true if the update is successful, false otherwise

CoffeeOrder:

- **addCoffee(coffee: Coffee): void** - Adds a Coffee object to the order
- **getTotal(): double** - Returns the total cost of the coffees in the order
- **printOrder(): String** - Generates and returns a string representation of the order

Coffee:

- **Coffee():** - Constructor for creating a Coffee object
- **setCoffeeType(type: CoffeeType): void** - Sets the type of coffee for the Coffee object
- **getCost(): double** - Returns the cost associated with the coffee type
- **printCoffee(): String** - Generates and returns a string representation of the coffee, including type and cost

AddOn:

- **setQuantity (quantity: int): void** - Sets the quantity of the add-on item
- **getType(): AddOnType** - Returns the type of add-on

Inventory:

- **readInventory(): Map<String, Integer>** - Reads inventory data from a file and returns current inventory status
- **writeInventory(): void** - Updates the inventory with the current status
- **updateOrderLog(): void** - Updates the order log with the latest order information

VERIFICATION TEST PLAN

The Unified Coffee Ordering System's software verification test plan is outlined in this document. The plan is designed to conduct exhaustive testing at the unit, integration, and system levels to ensure all features and functionalities meet the specified requirements.

SCOPE

This test plan targets the following features:

1. User Profile and Payment Management
 - a. Verification of individual functionalities within the 'User' class
 - b. Verification of payment-related operations within the 'PaymentInfo' class
2. Coffee Order Processing and Statistics Generation
 - a. Main functionalities like order logging and order processing
 - b. Statistical functionalities for coffee orders

TEST SET 1: USER PROFILE & PAYMENT MANAGEMENT

UNIT TEST - Targets individual functionalities or methods within specific classes

FEATURE TESTED: 'User' class: *getName[]* method, *getEmail()* method, *updateContactDetails()* method

TEST DESCRIPTION: Test each method with a variety of inputs, including boundary cases and invalid inputs to ensure proper handling and response. To do this, we will invoke *getName[]* after initializing a 'User' object with a predefined name to ensure the *getName[]* method returns the correct username.

TEST CASES:

- **INPUT:** To test the 'getName' function, we could set up a name for testing and use our 'getName' function.

- **EXPECTED OUTPUT:** We are expecting to get the name we set up as an output for this test case. If we obtain this, our function is working properly.
 - **Ex:** “John Smith” is returned when the method returns the correct user name (given this is the username set)

INTEGRATION TEST - Focuses on the interaction between closely related classes

FEATURE TESTED: ‘*PaymentInfo*’ class: ‘*verifyPayment[]*’ method & ‘*User*’ class

TEST DESCRIPTION: the interaction between the ‘*User*’ class and the ‘*PaymentInfo*’ class is checked, ensuring that the user’s payment information can be updated and verified successfully

TEST CASES:

- **INPUT:** To Test the interaction between the ‘*User*’ class and the ‘*PaymentInfo*’ class, we can use our ‘*verifyPayment*’ function to verify an existing account.
- **EXPECTED OUTPUT:** If the output of the following interaction of classes is true, we can infer that everything is working as expected.
 - **Ex:** ‘*verifyPayment*’ successfully returns true for our previous user “John Smith”.

SYSTEM TEST - Verify the software’s end-to-end workflow and overall functionality

TEST DESCRIPTION: the end-to-end process of a user updating their profile and payment details is verified

TEST CASES:

- **INPUT:** To verify that this process is working properly, we can update our contact details from our ‘*User*’ class, and then verify our payment from our ‘*PaymentInfo*’ class.

STEPS:

- Log in as “John Smith”

- Add payment information
- Verify payment information
- **EXPECTED OUTPUT:** If our ‘verifyPayment’ works after updating our contact details with our user class, we can ensure that the overall functionality of these classes is working correctly. This means the user profile updated, payment information has been added, and successfully verified.

TARGETED FAILURES:

- Incorrect retrieval or update of user details
- Failures in payment verification

TEST SET 2: COFFEE ORDER PROCESSING & STATISTICS GENERATION

UNIT TEST - Targets individual functionalities or methods within specific classes

FEATURE TESTED: ‘Statistics’ class: ‘*printStatistics[]*’ method

TEST DESCRIPTION: Testing the ‘*printStatistics*’ feature of the ‘Statistics’ class. This will print out a summary of all important statistics. This feature does not utilize other classes, making it a unit test.

TEST CASES:

- **INPUT:** Call *printStatistics()*
- **EXPECTED OUTCOME:** a list of popular drinks, total orders, and average order cost. If all statistics are printed correctly, we know that this feature works.

INTEGRATION TEST - Focuses on the interaction between closely related classes

FEATURE TESTED: ‘Statistics’ class: *getTotalOrders[]* method & ‘CoffeeOrder’ class

TEST DESCRIPTION: The system must process a certain number of orders from ‘CoffeeOrder’ class that is recorded. The function *getTotalOrders()* is then called.

TEST CASES:

- **INPUT:** Call *getTotalOrders()* after the system has processed the recorded number of orders
- **EXPECTED OUTCOME:** The number of orders returned matches the set number of orders decided earlier. Should be tested with values 1, 10, 100, and 1000 to cover the minimum to maximum expected orders that the coffee shop will receive in a day.

SYSTEM TEST - Verify the software's end-to-end workflow and overall functionality

TEST DESCRIPTION: the end-to-end process where a user places a coffee order and subsequently checks the coffee statistics

TEST CASES:

- **INPUT:** The user logs in, places a coffee order and then checks statistics.

STEPS:

- Log in as "Employee John"
- Place coffee orders received on a given day
- Check all coffee orders that have been received and logged
- **EXPECTED OUTCOME:** The total amount of coffee orders that have been received and logged – including coffee type, coffee add-ons, total order price, etc.

TARGETED FAILURES:

- Issues in adding coffee to orders
- Potential problems in the generation of coffee order statistics

DATA MANAGEMENT STRATEGY

IMPLEMENTATION RATIONALE:

For the Unified Coffee Ordering System, two essential databases were agreed upon for necessitating the operation of the software, the Statistics Database and the Inventory Database. For both databases, we have decided to utilize SQL for its consistency, relational data integrity, and complex query capabilities. These aspects of SQL align with the needs of an inventory and sales analytics system for the Unified Coffee Ordering System.

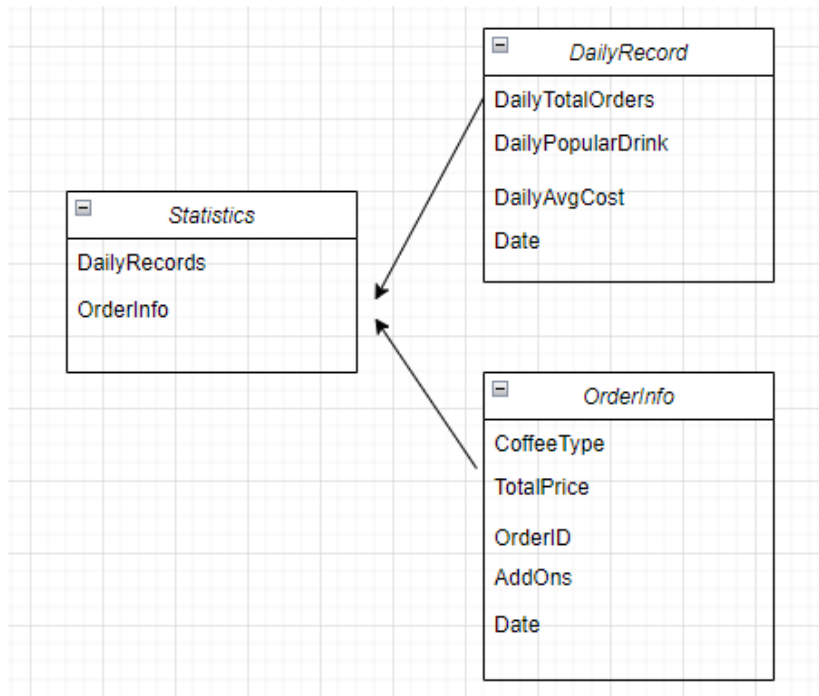
To elaborate, trustworthy transaction processing is ensured by the ACID (Atomicity, Consistency, Isolation, Durability) features of SQL databases. These qualities are essential in a setting where orders and payments need to be processed and documented with reliability. Furthermore, our software's data is naturally relational, and the UML diagram further supplements this notion. Orders relate to customers, inventory items, and possibly to employees or other personnel of interest. These relationships are handled naturally by SQL databases using an indexing method. The statistics database is a good example of this idea since it makes it simple to obtain particular data points that we might find useful. Finally, SQL databases perform at their best when structured data has a predefined schema. In the context of our software, an environment like a coffee shop where inventory items have specific attributes and prices that generally don't vary ensures data integrity and consistency.

STATISTICS DATABASE

DESCRIPTION:

The statistics database collects and stores data on sales, customer behavior, and even inventory turnover. It's meant to generate business insights through various reports such as sales trends, popular items, and peak times with total orders accumulated in a given day.

ENTITY RELATIONSHIP DIAGRAM:



EXAMPLE RETRIEVED DATA FROM MODELED TABLES:

ORDER INFO TABLE:

Data Type:	(Coffee)	(double)	(int)	(array)	(int)
Fields	Coffee Type	TotalPrice	OrderID	AddOns	Date
Example	latte	12.21	1234556	[1,2,3]	11092023

DAILY RECORD TABLE:

Date Type:	(int)	(Coffee)	(double)	(int)
Fields	DailyTotalOrders	DailyPopularDrink	DailyAverageCost	Date

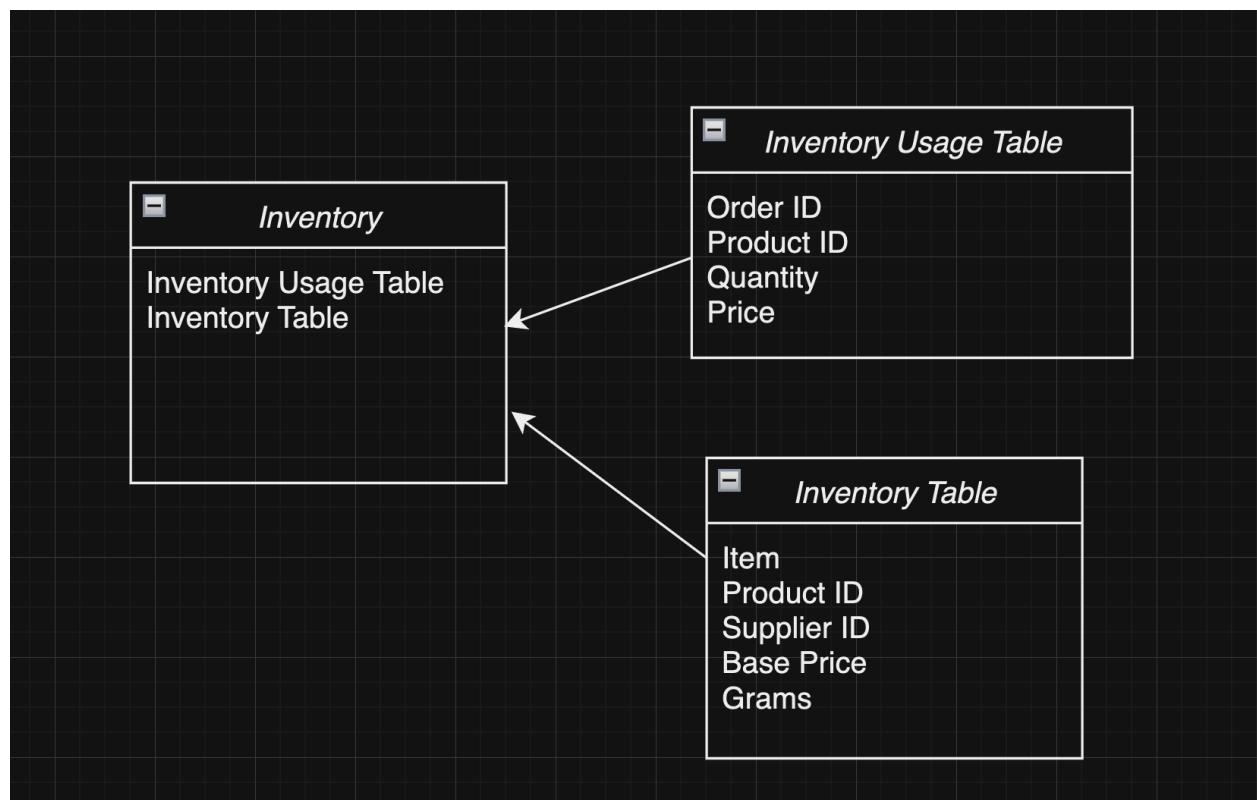
Example	345	Cappuccino	10.92	11102023
---------	-----	------------	-------	----------

INVENTORY/USAGE DATABASE

DESCRIPTION:

The inventory database manages all merchandise the coffee shop needs for daily functions – this includes items available for sale like coffee ingredients, add-ons, etc. It tracks the quantity for each item and updates in concurrence as orders are placed (specifically what items were utilized for said order) and as inventory is restocked.

ENTITY RELATIONSHIP DIAGRAM:



EXAMPLE RETRIEVED DATA FROM MODELED TABLES:

INVENTORY TABLE:

Data Type:	(String)	(int)	(String)	(int)	(int)
Fields:	Item	ProductID	SupplierID	Base Price	Grams

				per(500g)	
Examples	coffee beans	100	p11324123	12	390
	decaffeinated coffee beans	101	b12341321	16	330
	instant coffee	102	t12343214	10	250
	sugar	103	y12343211	5	500
	caramel	104	y12341234	8	156
	milk	105	p12754211	7	500
	syrup	106	k15435879	6	290
	whipped cream	107	j10234123	9	150
	cocoa	108	r12374531	11	150
	chai	109	m1976575	9	120

INVENTORY USAGE TABLE:

Data Type:	(int)	(int)	(int)	(double)
Fields:	OrderID	ProductID	Quantity	Price
Examples	1	100	2	12
√	2	101	1	16
	3	102	1	10
	4	107	4	9
	5	109	9	9

TRADE-OFF DISCUSSION

As we have discussed earlier, we have opted for an SQL database for our two databases, the Statistics Database and the Inventory Database. Splitting the inventory and statistics allows for optimization related to its specific usage patterns. The Inventory Database focuses on the

current state of data as inventory and is congruent with the usage of ingredients that are constantly changing with each coffee order inputted. This means that this database is transactional and requires high consistency. On the other hand, the Statistics Database is business-logic-heavy and optimized for historical data analysis. By choosing multiple SQL databases, we are prioritizing transaction integrity and its query capabilities, which align with the intentions of creating a streamlined inventory and sales analytics system for the Unified Coffee Ordering System.

POSSIBLE ALTERNATIVES

One possible alternative to our data management system would be to use a NoSQL approach, instead of the SQL databases we used. If our theoretical coffee shop expands and opens more stores we may need to switch to a noSQL system to more effectively handle the large amount of data. Another possible alternative would be to use one SQL database instead of the two we are currently using. Theoretically, we could implement this right away; however, having the data stored in two different sections makes it easier to find specific data. This is important because we have many functions that will need access to the data stored. A single SQL database could be used to simplify management, but this is at a cost of potential performance issues, specifically if the reporting loading of the Statistics Database interferes with the transaction processing of the Inventory Database— and vice versa.

FUTURE MODIFICATIONS

While the Unified Coffee Ordering System does offer the solution to essential business problems highlighted by the client, there is always room for improvement. Considered future modifications and potential planned updates to the Unified Coffee Ordering System include:

- **Mobile Integration:** To improve accessibility, create a mobile application that lets users place orders while they're on the go and get alerts about promotions and orders.
- **Loyalty Program Integration:** Put in place a rewards program that keeps track of consumer purchases and provides incentives, like freebies or discounts, to entice customers to come back.
- **Supply Chain Automation:** To manage inventory more effectively, integrate with suppliers to receive real-time inventory updates and automated restocking requests based on predictive analytics.
- **Customer Feedback System:** Include a platform in the system where customers can comment on their orders. This information can be utilized to improve customer service and foster direct communication.
- **Multilingual Support:** Provide multilingual assistance to staff members and customers who do not speak English to reach a worldwide audience.

SECURITY ANALYSIS

While the Unified Coffee Ordering System (UCOS) is fundamentally solid in its structure, it's critical to evaluate potential security threats and weaknesses in any system that handles sensitive data. Based on the CIA (Confidentiality, Integrity, Availability) classification, this examination looks at the security elements and suggests countermeasures to reduce risks.

CONFIDENTIALITY ANALYSIS

Potential Threats/Vulnerabilities:

- Unauthorized access to financial and customer information.
- Unauthorized tracking of data during transmission
- Internal risks include workers accessing private data without authorization

Countermeasures:

- Anti-malware, antivirus, and anti-keylogging software will be installed across the system
- For sensitive data, use encrypted standard practices
- Enforce strict access controls and role-based access to limit data exposure
- Conduct regular audits and monitoring to detect unauthorized access

INTEGRITY ANALYSIS

Potential Threats/Vulnerabilities:

- Order information and financial records could be changed by data manipulation
- Malware or malicious code that could corrupt data integrity

Countermeasures:

- Implement digital signatures to authenticate the source of data.
- Utilize secure coding practices to minimize vulnerabilities.
- Regularly update software to patch known security flaws.

AVAILABILITY ANALYSIS

Potential Threats/Vulnerabilities:

- Denial-of-service (DoS) attacks that could disrupt service availability
- Hardware failures or software crashes that could cause downtime
- Network issues that could prevent access to the system

Countermeasures:

- Deploy redundant systems and backup solutions to ensure uptime.
- Firewalls and intrusion detection/prevention systems will monitor and protect network traffic
- Implement failover strategies and load balancing to maintain service during high traffic.

USER MANUAL

SYSTEM DESCRIPTION

Welcome to the Unified Coffee Ordering System (UCOS), a cutting-edge software program created to improve business analytics, expedite operations, and improve the coffee ordering experience for global coffee providers. From basic system setup to advanced usage, this user handbook serves as your comprehensive guide to anything related to the software.

APPLICATION FUNCTIONALITY

PLACING AND CUSTOMIZING ORDERS

Navigate to the Customer Ordering Interface to select base coffees and add-ons.

Follow these steps:

1. Choose your base coffee from the menu.
2. Select any additional add-ons, such as cream or syrup.
3. View your customized coffee summary and confirm your order.

PAYMENT AND INFORMATION MANAGEMENT

Securely enter and verify payment information through the PaymentInfo class:

1. Input your card number, expiration date, and CVV.
2. Submit the payment information for verification.
3. Once verified, proceed to complete your order.

ORDER MANAGEMENT FOR EMPLOYEES

Access the Employee Inventory/Order Interface to manage orders:

1. View pending orders and their details.
2. Update order status from pending to complete.
3. Manage order cancellations and modifications.

ERROR REPORTING AND LOGGING

Report system issues through the Error Reporting Interface:

1. Describe the issue encountered in detail.
2. Submit the error report.
3. The Logger class will record the issue for system administrators.

ACCESSING ANALYTICS

Employees can view real-time business insights via the Employee Analytics Interface:

1. Log in to access dashboard features.
2. Select the type of report needed: sales, drink popularity, or inventory status.
3. Analyze the data to make informed business decisions.

INTERPRETING REPORTS

Understand and utilize statistical data for strategic planning:

1. Use the Statistics class to view historical data trends.
2. Identify top-selling coffees and peak sales hours.

INVENTORY MANAGEMENT

The Inventory class provides tools to track and manage stock levels:

1. Review current inventory levels.
2. Update stock quantities after each order fulfillment.
3. Generate inventory reports for reordering purposes.

SYSTEM REQUIREMENTS

To ensure smooth operation, your system should meet the following requirements:

- **Operating System:** *Windows 10 or higher, macOS X or higher*
- **Processor:** *Intel Core i5 or equivalent*
- **Memory:** *8 GB RAM*
- **Storage:** *500 MB of free disk space*
- **Internet Connection:** *Broadband with at least 10 Mbps download and upload speeds*

INSTALLATION GUIDE

Follow these steps to install UCOS:

1. Download the UCOS installer from the provided link.
2. Run the installer and follow the on-screen instructions.
3. Upon completion, launch the UCOS application.
4. Enter your provided credentials to access the system.

LIFE-CYCLE MODEL REFLECTION

DESCRIPTION OF LIFECYCLE

When I created the actual application of the Unified Coffee Ordering System, an Evolutionary Prototyping Model was used for the coding project, but as we have mentioned in previous classes, this type of life cycle model will typically result in a “sloppy” result. While my finished project wasn’t sloppy per se, it did need a good refinement and cleanup. If I were to implement this software in a professional and real-world scenario, I would choose the Rational Unified Process (RUP) as the software development life-cycle model for the Unified Coffee Ordering System. RUP is an incremental and iterative framework that prioritizes careful planning and a modular strategy for developing software. From a stakeholder’s perspective, this lifecycle model’s comprehensive and meticulous planning is a great measure in making sure the project is coming to fruition, under the timeframe given. Breaking down this software development project, it did feel like we followed the process of a typical RUP model.

- **Inception:** The first phase of the RUP model is inception which is the initial phase focused on establishing the project's scope and feasibility. We did this when we created our project groups, and we had to choose what problem statement to choose from. We then developed our first iteration of the software design specification where key objectives, including system functionality, project risks, cost estimates, and project schedule, were defined.
- **Elaboration:** The next phase of the RUP model is elaboration where the project's architecture is further refined. I would say this went in hand when we did our first software design specification, but it was more in-tune when we did our next iteration. We made a lot of changes related to our UML diagram as the requirements of the problem

statement were turned into a detailed design to address the necessary features and system behavior. We then included the foundation of how our databases would be implemented.

- **Construction:** The next stage of the RUP model is developing the software. In my previous class, the system was constructed and then repeatedly improved during development cycles. Coding, testing, and integration tasks were part of every iteration, and ongoing feedback shaped the work on further iterations. Having done just the coding aspect of the software in a different class, it was insightful to have the full picture of the entire life-cycle model of a given project.
- **Transition:** The last stage of the RUP model is moving the development to production of the software. This covered system deployment, training, and final user testing which we did when we created the test plan for our software. Although this was not commercially released, we did “distribute” the software through Github. If needed, any necessary adjustments could be made after distribution, as to guarantee that UCOS satisfied all user requirements and operational standards.

EFFECTIVENESS AND POTENTIAL MODIFICATIONS

As I have stated before, the RUP life-cycle model will most definitely work well for the UCOS project. Since this model type is iterative, this will allow for flexibility in design and development while retaining an organized approach. With constant user feedback and risk management integrated throughout the development process, the process made it easier to accommodate changing requirements and guaranteed a high-quality software output; however, scalability, and response to abrupt changes can be a hindrance by RUP's rigorous and documentation-heavy planning approach. This can be an issue where there is a need for

fast-paced work settings, or perhaps a change in deadline being pushed forward. Some potential modifications I can see being implemented that would mitigate some concerns are:

- **Scalability Consideration:** To ensure that the design can accommodate increasing user counts and data volumes, we could implement scalability checkpoints after each iteration
- **Enhanced Feedback Loops:** Holding user and stakeholder input sessions more frequently could guarantee that product development stays in line with consumer demands and industry developments
- **Risk Management Reassessment:** Regularly evaluating risks and creating a more adaptable mitigation plan may make it easier to deal with unforeseen difficulties